

Quản lý biến trong Python

Trịnh Tấn Đạt

Đại Học Sài Gòn

trinhtandat@sgu.edu.vn

<http://sites.google.com/site/ttdat88>

Nội dung

- Quản lý biến trong Python
- Namespace
- Truyền dữ liệu qua đối số hàm
- Dữ liệu khả biến và không khả biến

Mục tiêu học tập

Sau khi hoàn thành chương này, sinh viên có thể:

- Giải thích biến trong Python là tên tham chiếu đến object.
- Phân biệt `==` và `is`.
- Phân biệt alias, shallow copy và deep copy.
- Giải thích namespace và scope theo mô hình LEGB.
- Dùng đúng `global` và `nonlocal`.
- Phân tích sự khác nhau giữa truyền immutable và mutable object vào hàm.
- Tránh lỗi mutable default argument.
- Viết hàm không làm thay đổi dữ liệu đầu vào khi không cần thiết.

Biến là đối tượng

- Trong Python, biến là tên gắn với một object, không phải một “ô nhớ” độc lập chứa trực tiếp giá trị.

`pm25 = 32.5`

Tên: `pm25` -> Object float: 32.5

- Khi gán biến, Python tạo hoặc liên kết tên với object; hành vi của chương trình phụ thuộc vào object là khả biến hay không khả biến.

Tên biến, object và id()

```
a = 10
```

```
b = a
```

```
print(a)
```

```
print(b)
```

```
10
```

```
10
```

```
print(id(a))
```

```
127913992934448
```

```
print(id(b))
```

```
127913992934448
```

Thông thường a và b cùng tham chiếu đến object số nguyên 10.

```
print(a is b)    # True
```

```
print(a == b)    # True
```

== và is

```
a = [1, 2, 3]
b = [1, 2, 3]
c = a
```

```
print(a == b)
print(a is b)
```

```
print(a == c)
print(a is c)
```

```
True
False
True
True
```

Cặp biến	Giá trị giống nhau?	Cùng object?
a, b	Có	Không
a, c	Có	Có

Dùng == để so sánh dữ liệu.
Dùng is chủ yếu để kiểm tra singleton như None.

```
if value is None:
    print("Thiếu dữ liệu")
```

Rebinding: gán lại tên biến

```
a = 10  
b = a  
  
a = 20  
  
print(a) # 20  
print(b) # 10
```

Giải thích

- a = 20 không sửa object 10.
- a được gán lại với object mới 20.
- b vẫn tham chiếu object cũ 10.
- Đây là hành vi thường gặp với kiểu immutable như int, float, str, tuple.

Aliasing với dữ liệu khả biến

```
pm25_values = [12.5, 18.2, 35.1]
```

```
backup = pm25_values
```

```
backup.append(41.8)
```

```
print(pm25_values)
```

```
print(backup)
```

```
[12.5, 18.2, 35.1, 41.8]
```

```
[12.5, 18.2, 35.1, 41.8]
```

pm25_values —
| —> cùng một list object
backup —

Các kiểu dữ liệu immutable và mutable

Nhóm	Kiểu dữ liệu thường gặp	Có thể sửa tại chỗ?
Immutable	int, float, bool, str, tuple, frozenset, bytes	Không
Mutable	list, dict, set, bytearray	Có

```
name = "Python"
# name[0] = "J" # TypeError

values = [10, 20]
values[0] = 99

print(values)
```

Lưu ý: Tuple là immutable, nhưng nếu tuple chứa một list thì list bên trong vẫn mutable.

Copy list

```
original = [12.5, 18.2, 35.1]
```

```
copy_1 = original.copy()
```

```
copy_2 = list(original)
```

```
copy_3 = original[:]
```

```
copy_1.append(41.8)
```

```
print(original)
```

```
print(copy_1)
```

Lưu ý: Ba cách trên tạo shallow copy.

```
[12.5, 18.2, 35.1]
```

```
[12.5, 18.2, 35.1, 41.8]
```

Shallow copy và deep copy

```
import copy

original = [
    ["A01", 32.5],
    ["A02", 18.2]
]

shallow = original.copy()
deep = copy.deepcopy(original)

shallow[0][1] = 99.9

print(original)
print(shallow)
print(deep)
```

Nhận xét

- Shallow copy chỉ sao chép cấu trúc bên ngoài.
- Các object lồng nhau vẫn có thể được dùng chung.
- Deep copy sao chép đệ quy cả object bên trong.

```
[['A01', 99.9], ['A02', 18.2]]
[['A01', 99.9], ['A02', 18.2]]
[['A01', 32.5], ['A02', 18.2]]
```

Bài tập thực hành 1

- Bài 1. Dự đoán kết quả

```
a = [1, 2, 3]  
b = a  
c = a.copy()
```

```
b.append(4)  
c.append(5)
```

```
print(a)  
print(b)  
print(c)
```

```
print(a is b)  
print(a is c)  
print(a == c)
```

Bài tập thực hành 1

- **Bài 2. Dữ liệu lồng nhau**

```
records = [  
    ["A01", [12.5, 18.2]],  
    ["A02", [35.1, 28.4]]  
]
```

- Tạo shallow copy và deep copy.
- Sửa giá trị records[0][1][0].
- Quan sát sự khác nhau giữa ba biến.

Namespace và scope

Namespace là gì?

- Namespace là vùng lưu trữ ánh xạ giữa: Tên $\rightarrow$ Object
- Ví dụ:

```
station = "Tan Son Hoa"
```

```
pm25 = 32.5
```

Có thể hình dung namespace:

```
{  
  "station": "Tan Son Hoa",  
  "pm25": 32.5  
}
```

Các namespace thường gặp:

- Built-in namespace.
- Global namespace.
- Local namespace.
- Enclosing namespace.

Các namespace trong Python

Namespace	Được tạo khi	Ví dụ
Built-in	Khi Python khởi động	len, print, sum, range
Global	Khi module chạy	Biến khai báo ngoài hàm
Local	Khi một hàm được gọi	Biến trong hàm
Enclosing	Khi có hàm lồng nhau	Biến của hàm bao ngoài

```
x = "global"

def outer():
    y = "enclosing"

    def inner():
        z = "local"
        print(x, y, z)

    inner()
```

Quy tắc LEGB

- Python tìm kiếm tên biến theo thứ tự: $L \rightarrow E \rightarrow G \rightarrow B$

Ký hiệu	Tên	Ý nghĩa
L	Local	Phạm vi trong hàm hiện tại
E	Enclosing	Phạm vi của hàm bao ngoài
G	Global	Phạm vi module
B	Built-in	Tên dựng sẵn của Python

Quy tắc LEGB

```
x = "global"
```

```
def outer():  
    x = "enclosing"
```

```
    def inner():  
        x = "local"  
        print(x)
```

```
    inner()
```

```
outer()
```

local

Local variable và lỗi UnboundLocalError

```
count = 0
```

```
def increase():  
    count = count + 1  
    return count
```

- Khi gọi `increase()`, Python báo lỗi: `UnboundLocalError`

Nguyên nhân

- Vì trong hàm có phép gán `count = ....`
- Python xem `count` là biến local.
- Nhưng biến local này bị đọc trước khi được gán.

global

```
count = 0

def increase():
    global count
    count += 1
    return count

print(increase()) # 1
print(increase()) # 2
```

Khuyến nghị

- Có thể dùng global, nhưng không nên lạm dụng.
- Global state làm code khó kiểm thử, khó debug và khó tái sử dụng.
- Thường nên ưu tiên truyền dữ liệu vào hàm và trả kết quả ra ngoài.

nonlocal

```
def create_counter():  
    count = 0  
  
    def increase():  
        nonlocal count  
        count += 1  
        return count  
  
    return increase  
  
counter = create_counter()  
  
print(counter()) # 1  
print(counter()) # 2
```

Ý nghĩa

- nonlocal dùng cho biến trong phạm vi bao ngoài.
- Không áp dụng với global namespace.
- Rất quan trọng trong closure và decorator

Shadowing: che khuất tên có sẵn

```
list = [1, 2, 3]
```

```
numbers = list("123")
```

Dòng thứ hai sẽ lỗi vì tên list đã bị dùng cho biến.

- **Không nên đặt tên biến trùng với từ khóa**

locals() và globals()

```
station = "A01"

def show_namespace(pm25):
    temperature = 29.5
    print(locals())

show_namespace(32.5)

print(globals().keys())
```

Ý nghĩa

- locals() trả về mapping của local namespace hiện tại.
- globals() trả về mapping của global namespace hiện tại.
- Chủ yếu dùng để quan sát/debug, không nên dùng như cơ chế chính để điều khiển chương trình.

```
{'pm25': 32.5, 'temperature': 29.5}
dict_keys(['__name__', '__doc__', '__package__', '__loader__', '__spec__', '__annotations__', '__builtins__', '__file__', '__cached__', 'station', 'show_namespace'])
```

Bài tập thực hành 2

Bài 1. Dự đoán scope

- Dự đoán kết quả.
- Giải thích Python tìm biến theo thứ tự nào.

```
x = "global"

def outer():
    x = "outer"

    def inner():
        print(x)

    inner()

outer()
print(x)
```

Bài tập thực hành 2

Bài 2. Sửa lỗi

- Sửa code theo cách dùng global.
- Viết lại theo cách không dùng global.
- So sánh hai cách.

```
total = 0
```

```
def add_value(value):  
    total += value  
    return total
```

Truyền đối số và dữ liệu khả biến

Python truyền đối số như thế nào?

- Python truyền object reference vào hàm.
- Các thuật ngữ thường gặp:
 - Call by object reference.
 - Call by sharing.
 - Pass by assignment.
- Khi gọi hàm: `process(data)`

Python gán tên tham số trong hàm đến cùng object mà data đang tham chiếu.

Immutable object khi truyền vào hàm

```
def increase(value):  
    value += 1  
    print("Trong hàm:", value) # 11  
  
number = 10  
  
increase(number)  
  
print("Ngoài hàm:", number) #10
```

Giải thích

- int là immutable.
- value += 1 làm value tham chiếu object mới 11.
- Biến number bên ngoài vẫn tham chiếu object cũ 10.

Mutable object khi truyền vào hàm

```
def add_measurement(values, value):  
    values.append(value)  
    print("Trong hàm:", values)  
  
pm25_values = [12.5, 18.2]  
  
add_measurement(pm25_values, 35.1)  
  
print("Ngoài hàm:", pm25_values)
```

Giải thích

- values và pm25_values cùng tham chiếu một list.
- append() thay đổi object tại chỗ.
- Thay đổi được nhìn thấy cả trong và ngoài hàm.

Trong hàm: [12.5, 18.2, 35.1]

Ngoài hàm: [12.5, 18.2, 35.1]

Mutation và rebinding trong hàm

```
def change_list(values):  
    values.append(99)  
    values = [100, 200]  
    return values  
  
original = [10, 20]  
result = change_list(original)  
  
print("original:", original)  
print("result:", result)
```

original: [10, 20, 99]
result: [100, 200]

Dòng lệnh	Hành vi
values.append(99)	Mutation: sửa list gốc
values = [100, 200]	Rebinding: tên local trở sang list mới

Tránh side effect không mong muốn

```
def add_measurement_safe(values, value):  
    new_values = values.copy()  
    new_values.append(value)  
    return new_values
```

```
pm25_values = [12.5, 18.2]
```

```
updated_values = add_measurement_safe(pm25_values, 35.1)
```

```
print(pm25_values)
```

```
print(updated_values)
```

[12.5, 18.2]

[12.5, 18.2, 35.1]

Nguyên tắc

- Hàm xử lý dữ liệu nên nói rõ có thay đổi input hay không.
- Nếu không cần thiết, ưu tiên trả về object mới.
- Đặc biệt cẩn thận với list/dict lồng nhau.

Lỗi mutable default argument

```
def add_station(station, stations=[]):  
    stations.append(station)  
    return stations
```

```
print(add_station("A01"))  
print(add_station("A02"))  
print(add_station("A03"))
```

["A01"]

["A01", "A02"]

["A01", "A02", "A03"]

Nguyên nhân

- Default argument được tạo một lần khi định nghĩa hàm.
- List stations được tái sử dụng qua nhiều lần gọi hàm.

Cách xử lý mutable default argument

```
def add_station(station, stations=None):  
    if stations is None:  
        stations = []  
  
    stations.append(station)  
    return stations
```

```
print(add_station("A01"))  
print(add_station("A02"))  
print(add_station("A03"))
```

```
["A01"]  
["A02"]  
["A03"]
```

Lưu ý:

- Không dùng `[]`, `{}` hoặc `set()` làm giá trị mặc định nếu đối tượng sẽ bị thay đổi bên trong hàm.

Dictionary mutable trong hàm

```
def update_station(data, pm25):  
    data["pm25"] = pm25  
    data["status"] = "High" if pm25 > 25 else "Normal"  
  
station = {  
    "id": "A01",  
    "name": "Tan Son Hoa",  
    "pm25": 18.2  
}  
  
update_station(station, 35.1)  
  
print(station)
```

```
{'id': 'A01', 'name': 'Tan Son Hoa', 'pm25': 35.1, 'status': 'High'}
```

Copy dictionary an toàn

```
def update_station_safe(data, pm25):
    new_data = data.copy()

    new_data["pm25"] = pm25
    new_data["status"] = (
        "High" if pm25 > 25 else "Normal"
    )
    return new_data

station = {
    "id": "A01",
    "name": "Tan Son Hoa",
    "pm25": 18.2
}

updated_station = update_station_safe(station, 35.1)
print(station)
print(updated_station)
```

`{'id': 'A01', 'name': 'Tan Son Hoa', 'pm25': 18.2}`

`{'id': 'A01', 'name': 'Tan Son Hoa', 'pm25': 35.1, 'status': 'High'}`

Bài tập thực hành 3

- Bài 1. Mutation hay rebinding?

Dự đoán kết quả:

```
def process(values):  
    values.append(100)  
    values = values + [200]  
    return values
```

```
data = [1, 2, 3]
```

```
result = process(data)
```

```
print(data)  
print(result)
```

Bài tập thực hành 3

- Bài 2. Sửa hàm

```
def add_value(value, values=[]):  
    values.append(value)  
    return values
```

- Chứng minh hàm có lỗi.
- Sửa hàm.
- Viết thêm 3 test case.

Bài tập tổng hợp

Bài toán: Cập nhật dữ liệu trạm quan trắc an toàn

Cho dữ liệu:

```
station = {  
    "station_id": "A01",  
    "station_name": "Tan Son Hoa",  
    "measurements": [12.5, 18.2, 35.1]  
}
```

Viết các hàm:

```
add_measurement_inplace(station, value)  
add_measurement_safe(station, value)  
calculate_mean(station)  
create_station_counter()
```

Bài tập tổng hợp

Yêu cầu:

- `add_measurement_inplace` thay đổi trực tiếp dictionary gốc.
- `add_measurement_safe` trả về một dictionary mới, không thay đổi station gốc.
- Phân biệt shallow copy và deep copy trong trường hợp measurements là list lồng bên trong dictionary.
- `calculate_mean` không được thay đổi dữ liệu.
- `create_station_counter` dùng closure và nonlocal để đếm số lần cập nhật.
- Viết docstring cho tất cả hàm.
- Dùng ít nhất 5 lệnh `assert` để kiểm thử.

Tổng kết Chương 4

- Trong Python, biến là tên tham chiếu đến object.
- Cần phân biệt so sánh giá trị (==) và đồng nhất object (is).
- Dữ liệu mutable có thể gây aliasing và side effect.
- Copy nông phù hợp khi object không lồng nhau; deep copy phù hợp khi cần tách biệt hoàn toàn dữ liệu lồng nhau.
- Namespace tổ chức ánh xạ tên-object; Python tra cứu tên theo LEGB.
- global và nonlocal hỗ trợ thay đổi biến ngoài scope hiện tại nhưng nên dùng thận trọng.
- Python truyền object reference vào hàm; mutation có thể làm thay đổi dữ liệu phía ngoài.
- Tránh mutable default argument bằng mẫu None rồi khởi tạo bên trong hàm.
- Hiểu quản lý biến giúp viết code ổn định, dễ kiểm thử và ít lỗi khó phát hiện hơn.

Bài tập

- Cho dữ liệu của hai trạm quan trắc:

```
stations = {  
  "A01": {  
    "name": "Trạm A",  
    "measurements": [12.5, -999, 18.2]  
  },  
  "B02": {  
    "name": "Trạm B",  
    "measurements": [25.0, 30.1]  
  }  
}
```

Bài tập

Quy ước trong bài: -999 là giá trị thiếu; PM2.5 hợp lệ phải là số không âm. Hãy viết ba hàm sau:

- `clean_measurements(values)`: trả về list mới chỉ gồm các giá trị hợp lệ; không sửa `values`.
- `update_station_safe(data, station_id, value)`: trả về cấu trúc dữ liệu mới sau khi thêm một phép đo vào trạm tương ứng; không sửa `data`, kể cả các list lồng bên trong. Nếu không có `station_id` hoặc `value` không hợp lệ, phát sinh ngoại lệ phù hợp.
- `make_update_counter()`: trả về một hàm đếm số lần cập nhật thành công. Dùng `nonlocal`, không dùng biến toàn cục.

Sau đó viết đoạn chương trình: làm sạch phép đo của từng trạm, thêm giá trị 22.4 cho trạm "A01", tính trung bình PM2.5 của trạm này, và in số lần cập nhật thành công. Mỗi hàm cần có docstring.